1. What is React?

Answer: React is a JavaScript **library** used for **building user interfaces**, especially for **single-page applications**. It was developed by Facebook and follows a **component-based architecture**. It allows developers to create **reusable UI components**. It also improves **performance** with **efficient rendering.**

---

2. What are the key features of React?

Answer: Key features of React include:

- **JSX Format**: Allows writing HTML-like code inside JavaScript.
- **Components**: Reusable UI parts
- **Virtual DOM**: A lightweight copy of the real DOM used to optimize UI updates.
- **Unidirectional Data Flow**: Data flows in one direction from parent to child components via props.
- **React Hooks**: Used to manage state and lifecycle

---

3. What is JSX in React?

Answer:

- JSX (JavaScript XML) is a syntax in React that allows us to write HTML-like code inside JavaScript.
  It is used to create and design UI components easily.
  JSX is not understood by the browser, so it is converted into JavaScript using Babel.

Example:

jsx

Copy code

```
const element = <h1>Hello, world!</h1>;
```

---

4. What is a component in React?

Answer: A component is **a building block of a React application**. A component can either be a **class component** or a **functional component**, and it manages the **rendering of the UI based on input** (via props and state).

- **Functional Component**: Simpler, written as functions, can use hooks to manage state.
- **Class Component**: Has lifecycle methods and internal state (pre-hooks).

---

5. What is the difference between a class component and a functional component?

Answer:

- **Class Component**: Class Component is a React component written using **ES6 class**, which extends **React.Component** and supports **lifecycle methods** like **componentDidMount** and **componentWillUnmount.**

jsx

Copy code

```
class MyComponent extends React.Component {
  render() {
    return <h1>{this.props.name}</h1>;
  }
}
```

- **Functional Component**: Functional Component is a **simple JavaScript function** that **returns JSX** and **uses Hooks** to manage **state and side effects.**

jsx

Copy code

```
function MyComponent(props) {
  return <h1>{props.name}</h1>;
}
```

---

6. What are props in React?

Answer: Props (short for properties) are **read-only data** passed from a **parent component to a child component.** Props are **immutable** and **cannot be changed by the child component**. They are used to configure or **pass dynamic content**.

jsx

Copy code

```
function Greeting(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

---

7. What is state in React?

Answer: State is a **built-in object in react** used to **store data that can change over time**.

- It controls how the **component behaves** and what it displays.
- When **state changes**, the component **automatically re-renders.jsx**

Copy code

```
const [count, setCount] = useState(0);
```

---

8. What are React Hooks?

Answer: React Hooks are **functions** introduced **in React 16.8** that enable functional components to **manage state and side effects**. The most common hooks are:

- **useState**: To manage state in functional components.
- **useEffect**: To perform side effects such as data fetching or DOM manipulation.
- **useContext**: For accessing values from context without prop drilling.
- **useReducer**: For managing more complex state logic.

---

9. What is useState in React?

Answer: useState is a hook that allows **functional components** to **declare, store, and update state variables**, enabling the component to **re-render whenever the state changes**.

jsx

Copy code

```
const [count, setCount] = useState(0); // Initializes state with value 0
```

---

10. What is useEffect in React?

Answer: useEffect is a hook that **performs side effects** in functional components (e.g., data fetching, subscriptions, DOM updates). It runs after **every render by default**, but you can control when it runs using a **dependency array**.

It can be used for lifecycle methods like componentDidMount, componentDidUpdate, and componentWillUnmount in class components.

jsx

Copy code

```
useEffect(() => {
  // Code to fetch data
  console.log('Component mounted');
}, []); // Empty dependency array ensures it runs only once (on mount)
```

---

11. What is the Virtual DOM in React?

Answer: The Virtual DOM is a lightweight copy of the **actual DOM**. React compares changes using a **diffing algorithm**. Only the **changed parts** are **updated** in the **actual DOM**. This improves performance significantly.

---

12. What is React Context?

Answer: React Context is used for **global state management**.

- It helps **avoid prop drilling** by sharing **data** across components.
- It uses Provider and Consumer.
- It is useful for **themes, authentication,language preferences**.

jsx

Copy code

```
const ThemeContext = React.createContext('light');
```

---

13. What is Redux?

Answer: Redux is a **state management library** used for **managing global state**.

- It uses a **centralized store** to **hold data.**
- State changes are made using actions and reducers.
- It provides predictable state updates.

---

14. What is the connect() function in React Redux?

Answer: connect() is a function used to connect **React components to the Redux store**.

- It allows components to **access store data** via mapStateToProps and **dispatch actions** via mapDispatchToProps.

jsx

Copy code

```
const mapStateToProps = (state) => ({
  count: state.count
});


const mapDispatchToProps = (dispatch) => ({
  increment: () => dispatch({ type: 'INCREMENT' })
});
```

export default connect(mapStateToProps, mapDispatchToProps)(MyComponent);

---

15. What are controlled components?

Answer: Controlled components are **form elements controlled by React state**.

- The input value is **managed using state**.
- They provide better control and validation.
- They are commonly used in forms.

jsx

Copy code

```
const [value, setValue] = useState('');

return <input value={value} onChange={(e) => setValue(e.target.value)} />;
```

---

16. What are uncontrolled components?

Answer: An uncontrolled component is an element where the **form data is handled by the DOM itself, not by React**. Refs are used to interact with the DOM and get the value of the input.

jsx

Copy code

```
const inputRef = useRef(null);

<input ref={inputRef} />;
```

---

17. What is Prop Drilling in React?

Answer: Prop drilling refers to the process of **passing data** from a **parent component** down to deeply **nested child components** through intermediate levels of components. This can make the code **harder to maintain**.

Solution: Use **React Context** or **Redux** to avoid prop drilling by passing data globally to all components.

---

18. What are keys in React?

Answer: Keys are **unique identifiers** used in lists. They **help React identify elements efficiently**. They **improve performance** during **rendering**. Avoid using index as a key when possible.

jsx

Copy code

```
const listItems = items.map((item, index) => <li key={index}>{item}</li>);
```

---

19. What are lifecycle methods in React?

Answer Lifecycle methods are functions in **class components** that run at **different stages** of a **component's life** such as **mounting, updating, and unmounting**.

Common lifecycle methods include:

- componentDidMount
- componentDidUpdate
- componentWillUnmount

With functional components, the useEffect hook is used to manage side effects.

---

20. What is React Router?

- **Answer**: React Router is a library used for navigation in React applications.
- It enables routing without reloading the page.
- It is useful for single-page applications.
- It provides components like Route and Link.

jsx

Copy code

```
import { BrowserRouter as Router, Route } from 'react-router-dom';


<Router>
  <Route path="/home" component={Home} />
  <Route path="/about" component={About} />
</Router>
```

---

21. What is shouldComponentUpdate in React?

Answer: shouldComponentUpdate is used to **control re-rendering in class components**. It improves performance by **preventing unnecessary updates**. It returns **true or false**. It is **useful** for **optimization.**

---

22. What is React.memo()?

Answer: React.memo is used **to prevent unnecessary re-renders** in functional components.

- It compares **previous and current props**. It improves performance.
- It works like shouldComponentUpdate.

jsx

Copy code

```
const MyComponent = React.memo(function MyComponent(props) {
  // Component code here
});
```

---

23. What is useCallback in React?

Answer: useCallback is a hook that **returns a memoized version** of a **callback function**, which only **changes** if one of its **dependencies has changed**. It helps to **avoid unnecessary re-creations** of **functions.**

jsx

Copy code

```
const memoizedCallback = useCallback(() => {
  // Code here
}, [dependency]);
```

---

24. What is useMemo in React?

Answer: useMemo is a hook that **memoizes the result** of a **computation**. It **only recomputes** the value when **one of its dependencies changes**, optimizing performance for expensive calculations.

jsx

Copy code

```
const memoizedValue = useMemo(() => expensiveFunction(count), [count]);
```

---

25. What are fragments in React?

Answer Fragments allow grouping multiple elements without adding extra DOM nodes. They keep the DOM clean. They improve structure.

jsx

Copy code

```
return (
  <>
    <h1>Title</h1>
    <p>Some content</p>
```

```
  </>
);
```

---

26. What is the difference between componentDidMount and useEffect?

Answer:

- componentDidMount is a **lifecycle method i**n **class components** that is called **once after the component is mounted**.
- useEffect can be used in **functional components** to perform **side effects**, and it **runs after every render**, but can be configured to run only once or when specific dependencies change.

---

27. What is the dangerouslySetInnerHTML attribute in React?

Answer: dangerouslySetInnerHTML is a **React attribute** that allows you to set **HTML content directly in a component**. It's called "dangerous" because it can lead to XSS (Cross-Site Scripting) vulnerabilities if you inject unsanitized user input.

jsx

Copy code

```
<div dangerouslySetInnerHTML={{ __html: 'Hello <b>World</b>' }} />
```

---

28. What is Server-Side Rendering (SSR) in React?

Answer: SSR is the process of **rendering React components** on the server and **sending the fully rendered page to the client**, which improves performance and SEO. Tools like **Next.js** make it easy to implement SSR in React.

---

29. What is Client-Side Rendering (CSR) in React?

Answer: CSR is the process where the **browser fetches the JavaScript** and **renders the React components** on the client side. React is first loaded, and only then is the **UI generated.** CSR is typical in single-page applications (SPAs).

---

30. What is the purpose of useRef in React?

Answer: useRef is a hook used to **store a mutable value** that **does not cause re-render when it changes** and to **directly access DOM elements**.

useRef is used to access DOM elements directly. It stores mutable values without causing re-renders.

jsx

Copy code

```
const inputRef = useRef();
```

---

31. How do you handle forms in React?

Answer: Forms in React are typically controlled components where form data is stored in the state, and changes are handled using event handlers like onChange and onSubmit.

jsx

Copy code

```
const [value, setValue] = useState('');


function handleChange(event) {
  setValue(event.target.value);
}


return <input value={value} onChange={handleChange} />;
```

---

32. What is a Higher-Order Component (HOC)?

Answer: An HOC is a function that takes a component and returns a new component with additional props or functionality. HOCs are used for cross-cutting concerns like authentication, logging, etc.

jsx

Copy code

```
function withLogging(Component) {
  return function WrappedComponent(props) {
    console.log('Rendering component', Component.name);
    return <Component {...props} />;
  };
}
```

1. What is React Fiber?
React Fiber is the new reconciliation engine introduced in React to improve rendering performance. It enables incremental rendering, meaning React can split work into smaller units and prioritize tasks. This helps in smoother UI updates and better responsiveness. It also supports features like suspense and concurrent rendering.

---

2. What is code splitting?
Code splitting is a technique used to split a large bundle into smaller chunks. It helps load only the required parts of the application when needed. This improves performance and reduces initial loading time. In React, it is commonly implemented using React.lazy and dynamic import.

---

3. What is React Strict Mode?
React Strict Mode is a tool for highlighting potential problems in an application. It helps identify unsafe lifecycle methods and deprecated APIs. It runs only in development mode and does not affect production. It improves code quality and best practices.

---

4. How does React handle events?
React uses a synthetic event system to handle events consistently across browsers. Events are named using camelCase like onClick instead of onclick. Event handlers are passed as functions. React manages event delegation internally for better performance.

---

5. What is conditional rendering?
Conditional rendering means displaying UI elements based on conditions. It is similar to JavaScript conditions like if, ternary operator, or && operator. It helps show or hide components dynamically. It improves user experience by rendering relevant content.

---

6. How do you handle errors in React?
Errors in React are handled using Error Boundaries. These are special components that catch JavaScript errors in child components. They prevent the whole app from crashing and show fallback UI. For async errors, try-catch or error handling in APIs is used.